

ด้านที่ 1

ภารกิจที่ 1: ตรวจสอบความถูกต้องของไฟล์ (Hash Integrity)

- โจทย์ถามว่า: อัลกอริทึมที่ใช้ทำ "Hashing" เพื่อเช็คความถูกต้องของไฟล์ payload.bin คืออะไร?
 - วิธีหาคำตอบ: 1. เปิดไฟล์ manifest.json
2. มองหาบรรทัดที่เขียนว่า "friendly_name": "SHA256" หรือดูเลข OID ที่ขึ้นต้นด้วย 2.16.840...
 3. คำตอบคือ: SHA256
-

ภารกิจที่ 2: กุญแจสาธารณะ RSA (Asymmetric Crypto)

- โจทย์ถามว่า: ในไฟล์ใบรับรอง cert.hex ค่า Public Exponent (e) ในรูปแบบเลขฐานสิบหก (Hex) คืออะไร?
 - วิธีหาคำตอบ: 1. เปิดไฟล์ cert.hex
2. มองหาบรรทัดต่างๆ ที่มีคอมเมนต์ว่า <-- RSA_E (65537)
 3. คุณจะเห็นเลข 02 03 01 00 01 ซึ่งเลข 01 00 01 ก็คือค่าคงที่มาตรฐานของ RSA
 4. คำตอบคือ: 0x10001 (ต้องเติม 0x นำหน้าตามที่โจทย์สั่ง)
-

ภารกิจที่ 3: กุญแจลับ AES (Symmetric Crypto)

- โจทย์ถามว่า: ไปหา "รหัสลับ 32 ไบต์" (Key) ที่ซ่อนอยู่ในไฟล์หน่วยความจำ mem_dump.bin มาหน่อย
 - วิธีหาคำตอบ: 1. เปิดไฟล์ mem_dump.bin
2. มองหาคำว่า SKEY: (ย่อจาก Symmetric Key)
 3. ก๊อปปี้ตัวเลขยาวๆ ที่อยู่ข้างหลังมาทั้งหมด
 4. คำตอบคือ: A5B2C3D4E5F67890A1B2C3D4E5F67890

ภารกิจที่ 4: คำนวณรหัสลับร่วม (Diffie-Hellman Secret)

- โจทย์ถามว่า: ถ้ากำหนดค่ามาให้ดังนี้ จงคำนวณหา "Shared Secret" (ผลลัพธ์ของการแลกเปลี่ยนกุญแจ)

- $p = 23$ (เลขเฉพาะ)
- $g = 5$ (ตัวสร้าง)
- $A = 8$ (กุญแจสาธารณะของฝั่ง Client)
- $b = 3$ (กุญแจลับของฝั่ง Server)

- วิธีหาคำตอบ: 1. ใช้สูตร: $(A^b \bmod p)$

2. แทนค่า: $(8^3 \bmod 23)$

3. คำนวณ: $8 \times 8 \times 8 = 512$

4. เอา 512 หารด้วย 23 แล้วดู "เศษ" ที่เหลือ (ใช้เครื่องคิดเลขกด $512 \% 23$)

5. คำตอบคือ: 6

ภารกิจที่ 5: ลายเซ็นดิจิทัล (Digital Signature)

- โจทย์ถามว่า: ผู้บุกรุกเซ็นชื่อทิ้งไว้ท้ายไฟล์ จงหาข้อความ "Flag" ที่อยู่ล่างสุดของบล็อกลายเซ็น

- วิธีหาคำตอบ: 1. เปิดไฟล์ handshake.txt

2. เลื่อนลงไปล่างสุดของไฟล์ มองหาบรรทัดที่เขียนว่า EOF_MARKER:

3. คำตอบคือ: FLAG{CRYPTO_M4STER_091}

ด้านที่ 2

เฉลยสำหรับการผ่านด่าน **Stage 2: Authentication** ในโค้ดนี้มี 2 ส่วนหลักที่คุณต้องทำ คือการใส่ OTP และการหา Flag ครบ โดยคำใบ้ทั้งหมดซ่อนอยู่ใน artifactData (ไฟล์เอกสารจำลอง) ดังนี้ครับ:

1. วิธีแก้ปริศนา OTP (Interactive OTP)

ในส่วนของ otp_seed.txt มีหมายเหตุระบุไว้ว่า:

"The emergency fallback code for analyst_07 is always the current challenge token padded with zeros if needed."

- **โจทย์บนหน้าจอ:** Challenge Token คือ 49-X-91
 - **วิธีคิด:** นำตัวเลขจาก Token มาเรียงกันให้ครบ 6 หลักตามที่ระบบต้องการ
 - **คำตอบ:** 490091 (หรือหากระบบต้องการเพียงการตัดอักษรออกจะเป็น 4991 แล้วเติม 0 ให้ครบ 6 หลัก ซึ่งโดยทั่วไปใน logic นี้คือ 490091)
 - **หมายเหตุ:** ในโค้ดปัจจุบันฟังก์ชัน handleVerifyOtp ตั้งค่าให้รับตัวเลข 6 หลักใดก็ได้เพื่อให้เกมลื่นไหล แต่ตามเนื้อเรื่องคือการใช้เลขจาก Challenge Token ครับ
-

2. วิธีหา Flag (Authentication Flag Input)

ในส่วนของ session_meta.json มีการบอกใบ้เรื่องช่องโหว่ของ JWT (JSON Web Token):

"The HS256 secret key was set to the default company name 'CYBERUNIT' during the initial setup."

และในส่วนของ handleVerifyFlag มีการเช็คค่าที่ถูกต้องไว้ในโค้ดคือ:

- **คำตอบ (Flag):** FLAG{JWT_WEAK_SIGN}
-

สรุปขั้นตอนการผ่านด่าน:

1. กรอก **490091** ในช่อง OTP แล้วกด **Verify Response**
2. เมื่อระบบแจ้งว่า MFA Step 1/2 สำเร็จ ให้ไปที่ช่อง Flag Input ด้านล่าง
3. พิมพ์ **FLAG{JWT_WEAK_SIGN}** แล้วกด **Verify Auth**

เกร็ดความรู้จากด่านนี้:

- **Weak JWT Secret:** การใช้ Key ที่เดาง่าย (เช่น ชื่อบริษัท CYBERUNIT) ทำให้ผู้ไม่หวังดีสามารถปลอมแปลงสิทธิ์ (Privilege Escalation) ใน Token ได้
- **Insecure MFA Fallback:** การตั้งรหัสสำรอง (Emergency Code) ที่อิงจากข้อมูลที่แสดงบนหน้าจอ (Challenge Token) ทำให้ระบบ MFA หมดความหมายเพราะใครก็ได้รหัสได้

ด้านที่ 3

สำหรับโค้ดในด้าน **Stage 3: Authorization** นี้ โจทย์มุ่งเน้นไปที่เรื่องของสิทธิ์การเข้าถึง (Privileges) และช่องโหว่ประเภท IDOR (Insecure Direct Object Reference) หรือ BOLA (Broken Object Level Authorization) ครับ

1. วิเคราะห์จากโค้ด (The Solution)

หากดูที่ฟังก์ชันการตรวจสอบ `handleVerify` ในไฟล์ที่คุณส่งมา จะเห็นคำตอบที่ถูกตั้งค่าไว้โดยตรงคือ:

Flag: `FLAG{ADMIN_IDOR_X}`

2. ที่มาและคำอธิบายของปริศนา (The Logic)

ในฐานะนักเจาะระบบ (หรือผู้เล่น) คุณสามารถหาคำตอบนี้ได้จากเบาะแสที่ซ่อนอยู่ใน UI components ดังนี้ครับ:

- **Role Tree & Rule Evaluation:**
 - ระบบแจ้งว่าคุณคือ Analyst (`usr_882910`) ซึ่งมีสิทธิ์แค่ Level 1
 - ในส่วน **Rule Evaluation** (ขวาล่าง) มีกฎข้อหนึ่งที่ FAIL คือ: `user.role == "admin"`
 - นี่คือการบอกว่าคุณต้องหาทาง "ปลอมตัว" หรือ "ข้ามสิทธิ์" ไปเป็น **Admin**
- **Terminal Logs (TTY3):**
 - `GET /api/v1/admin/flags [403 FORBIDDEN]`
 - ระบุว่าความพยายามเข้าถึงทรัพยากรของ Admin ถูกปฏิเสธเนื่องจาก **Insufficient Scope**
- **JWT Claims Analysis (กลางจอ):**
 - `"role": "analyst"` และ `"scope": "read:cases"`

- หากนี้เป็นสถานการณ์จริง ผู้เล่นจะต้องทำการ IDOR โดยการเปลี่ยน ID ใน Request หรือแก้ไข JWT Payload (ถ้า Signature ไม่แน่นอนหาพอ) เพื่อเปลี่ยน Role จาก analyst เป็น admin

3. สรุปวิธีการผ่านด่าน

1. สังเกตว่าเป้าหมายคือการเข้าถึง /api/v1/admin/flags
2. ดูรูปแบบของ Flag ที่ระบบต้องการในช่อง Input (FLAG{ADMIN_IDOR_...})
3. นำค่าไปเรื่องช่องโหว่ IDOR และสิทธิ์ Admin มาประกอบกัน
4. พิมพ์ FLAG{ADMIN_IDOR_X} ในช่อง "System Override Flag" แล้วกด Verify Authorization

เกร็ดความรู้เรื่องความปลอดภัย:

- IDOR (Insecure Direct Object Reference): เกิดขึ้นเมื่อแอปพลิเคชันเชื่อใจข้อมูลที่ส่งมาจากฝั่ง User (เช่น ID หรือ Role ใน Token) โดยไม่มีการตรวจสอบซ้ำที่ฝั่ง Server ว่า User คนนั้นมีสิทธิ์เข้าถึงข้อมูลของวัตถุ (Object) นั้นจริงหรือไม่
- Horizontal vs Vertical Privilege Escalation: * Horizontal: เข้าถึงข้อมูลของ User คนอื่นในระดับเดียวกัน
 - Vertical: ข้ามชั้นจาก User ธรรมดาขึ้นไปเป็น Admin (เหมือนในโจทย์ด้านนี้)